

Riassunto Lezione 1: Introduzione ai Paradigmi di Programmazione

Insegnamento: Paradigmi di Programmazione

Modulo: Programmazione Funzionale e OO

Docente: Prof. Giancarlo Ruffo (giancarlo.ruffo@uniupo.it)

1 Architettura dei Calcolatori e Astrazione Software

I calcolatori elettronici moderni si basano su principi derivati dall'architettura di von Neumann (1945), composta fondamentalmente da:

- **CPU (Central Processing Unit):** contenente registri, ALU (Arithmetic Logic Unit) e unita di controllo.
- **Memoria Principale:** destinata a ospitare dati e istruzioni.
- **Dispositivi di I/O (Input/Output):** per l'interazione con l'esterno.

Poiché l'hardware nativo comprende unicamente operazioni elementari (spostamento di dati tra registri, operazioni aritmetico-logiche in binario), lo sviluppo di applicazioni complesse richiede il concetto di **astrazione**. Il software viene organizzato in strati gerarchici:

1. **Hardware:** codice binario nativo, specifico dell'architettura.
2. **Sistema Operativo (S.O.):** gestisce l'hardware ed eroga servizi di base.
3. **Librerie ed Applicativi:** componenti software di alto livello.
4. **Interfaccia Utente (I.U.):** livello di interazione finale con l'utente.

1.1 API e Librerie

- **API (Application Programming Interface):** insieme di servizi e interfacce che un componente software mette a disposizione di altri moduli per consentirne l'interoperabilità (ad esempio, le chiamate di sistema messe a disposizione dal S.O. per la gestione dei file o della shell).
- **Librerie:** raccolte organizzate di istruzioni e funzioni ad uso frequente (es. gestione dei protocolli di rete o invio e-mail). Permettono di evitare la riscrittura di codice standard, garantiscono uniformità, riducono i bug e migliorano la manutenibilità.
Nota: Le librerie non sono programmi autonomi.

2 Classificazione dei Linguaggi di Programmazione

Attualmente esistono oltre 700 linguaggi di programmazione (di cui circa 245 attivamente utilizzati). La molteplicità di linguaggi è dovuta alla specializzazione per ambiti applicativi (web, analisi dati, sistemi embedded) e a fattori industriali/commerciali.

I linguaggi si dividono preliminarmente in tre livelli:

- **Linguaggio Macchina:** costituito da sequenze di cifre binarie (0 e 1), direttamente eseguibile dall'hardware e strettamente legato all'architettura fisica.
- **Linguaggio Assembly:** utilizza abbreviazioni mnemoniche per rappresentare le istruzioni macchina. Viene tradotto in codice macchina tramite un *assembler*.
- **Linguaggi ad Alto Livello:** utilizzano una sintassi vicina al linguaggio umano e alla notazione matematica. Richiedono traduzione tramite compilatori o interpreti.

2.1 Modelli di Esecuzione

I programmi scritti in linguaggi ad alto livello vengono eseguiti secondo due modalità principali:

Modello	Meccanismo	Vantaggi
Compilato	Un <i>compilatore</i> traduce l'intero sorgente in un eseguibile in codice macchina prima dell'esecuzione.	Alta efficienza, controllo a basso livello, ottimizzazione dell'hardware.
Interpretato	Un <i>interprete</i> analizza ed esegue il codice sorgente istruzione per istruzione a tempo di esecuzione.	Portabilità multiplatforma, maggiore sicurezza, facilità di debugging.

Table 1: Confronto tra Linguaggi Compilati e Interpretati

3 Paradigmi di Programmazione

Un paradigma definisce la filosofia e lo stile con cui il programmatore esprime la computazione.

3.1 Operazionale vs Dichiarativo

- **Linguaggio Operazionale/Procedurale:** il programmatore specifica *come* svolgere il compito, descrivendo la sequenza esatta di passaggi ed esecuzioni.
- **Linguaggio Dichiarativo:** il programmatore specifica *cosa* si vuole ottenere, lasciando all'esecutore/interprete la ricerca dell'algoritmo ottimale per il calcolo.

3.2 I Principali Paradigmi

1. **Imperativo / Procedurale:** focalizzato sulla sequenza di comandi da eseguire. La computazione è modellata tramite modifiche dello stato della memoria. L'operazione cardine è l'assegnamento di un valore a una variabile.
2. **Orientato agli Oggetti (OO):** la computazione avviene tramite lo scambio di messaggi tra entità autonome dette *oggetti*, le quali incapsulano uno stato interno e metodi per manipolarlo.
3. **Funzionale:** basato sul concetto matematico di funzione. La computazione è descritta dalla valutazione di espressioni e dall'applicazione di funzioni ad argomenti, evitando modifiche di stato ed effetti collaterali.

4. **Logico:** il programma è definito da un insieme di assiomi e regole di inferenza. La computazione coincide con la ricerca della dimostrazione formale di un'asserzione (es. Prolog).
5. **Guidato da Eventi (Event-Driven):** tipico dei sistemi IoT e interfacce grafiche; la computazione non segue un flusso monolitico ma si attiva in risposta asincrona ad eventi rilevati da sensori o azioni utente.

3.3 Esempio Comparativo: Calcolo del Fattoriale

Data la definizione matematica del fattoriale:

$$n! = \prod_{k=1}^n k = 1 \cdot 2 \cdot 3 \cdots (n-1) \cdot n \quad (1)$$

Di seguito la risoluzione nei diversi paradigmi:

Imperativo (Python):

```
def fact(n):
    risultato = 1
    for i in range(2, n + 1):
        risultato *= i
    return risultato
```

Funzionale (Python):

```
def fact(n):
    return 1 if n == 0 else n * fact(n - 1)
```

Logico (Prolog):

```
fact(0, 1).
fact(N, F) :- N > 0, N1 is N - 1, fact(N1, F1), F is N * F1.
% Interrogazione: ?- fact(5, F). -> F = 120
```

Orientato agli Oggetti (Python):

```
class Fact:
    def __init__(self, n):
        self.n = n
    def calcola(self):
        risultato = 1
        for i in range(2, self.n + 1):
            risultato *= i
        return risultato

f = Fact(5)
print(f.calcola())
```

Nota: I linguaggi di programmazione moderni (es. Python, Java, C++, Scala) sono prevalentemente *multi-paradigma*, consentendo l'integrazione di approcci diversi.

4 Ecosistema, Metodologie e Ciclo di Vita del Software

4.1 Metodologie e Architetture Software

- **Agile Software Development:** approccio metodologico che promuove lo sviluppo rapido, incrementale e flessibile riducendo i costi di processo.
- **Refactoring:** tecnica di ristrutturazione del codice sorgente volto a migliorarne leggibilità, manutenibilità ed eleganza senza alterare il comportamento esterno.
- **Design Pattern:** schemi risolutivi standardizzati e riutilizzabili per problemi ricorrenti di progettazione ad oggetti.
- **Stack LAMP / WAMP:** acronimo di Linux (o Windows), Apache, MySQL, PHP/Perl/Python, pila tecnologica open-source fondamentale per lo sviluppo di applicazioni web.
- **SaaS (Software as a Service):** fruizione del software distribuito e centralizzato sul cloud via browser web, eliminando la necessità di installazione locale e semplificando gli aggiornamenti.
- **PaaS (Platform as a Service):** erogazione via rete di una piattaforma completa di sviluppo, esecuzione ed hosting (es. Google App Engine, AWS EC2, Azure).
- **Cloud Computing:** architettura che fornisce risorse computazionali, storage e gestione infrastrutturale su richiesta ed elastica via Internet.
- **SDK (Software Development Kit):** pacchetto integrato di librerie, strumenti di compilazione e documentazione fornito per sviluppare applicazioni su una specifica piattaforma.

4.2 Ciclo di Vita e Versionamento del Software

1. **Alpha:** prima versione interna, incompleta e instabile, rilasciata a una cerchia ristretta di sviluppatori per test iniziali e rilevamento bug.
2. **Beta:** versione con funzionalità essenzialmente complete, distribuita ad un pubblico più ampio di utenti/tester per verificare la stabilità e raccogliere riscontri.
3. **Release Candidate (RC):** versione potenziale per la pubblicazione finale, testata su un'ampia varietà di configurazioni hardware/software.
4. **Release Finale:** versione ufficiale definitiva per la distribuzione generale.
5. **Beta Continua (Rolling Release):** modello evolutivo tipico delle applicazioni Web/Cloud (es. Gmail), prive di numeri di versione tradizionali e costantemente aggiornate lato server.